Architectural Governance and Machine-Readable Contextualization: A Technical Analysis of the Stela Platform and the Strategic Implementation of llms-full.txt

The rapid evolution of agentic systems in software engineering has necessitated the creation of governance frameworks that prioritize machine-readable explainability and rigorous provenance. The Stela platform, as evidenced by its repository structure and constitutional directives, represents a highly sophisticated attempt to solve the dual problems of system entropy and agentic drift in a complex content management system environment. By analyzing the current staffing protocols, behavioral logic standards, and the operational tools provided within the Stela repository, it is possible to assess the potential impact of emerging industry standards such as the llms.txt and llms-full.txt specifications. This report provides a comprehensive evaluation of Stela's existing architecture and theorizes the second- and third-order implications of integrating full-context machine-readable documentation.

The Constitutional Underpinnings of the Stela Platform

The Stela platform operates under a governance model that is fundamentally explainability-first and provenance-forward. This philosophy is codified in a set of canonical surfaces that establish the immutable laws of the system, the history of its changes, and the logic of its current operations. The core of this framework is the relationship between the authority of the human operator and the logic of the artificial intelligence agent, a relationship defined by strict jurisdictional boundaries and behavioral standards.[1]

Jurisdictional Silos and the Staffing Protocol

The repository's governance is anchored by the AGENTS.md file, which establishes a tripartite staffing protocol. This model distinguishes between the Operator, the Integrator, and the Contractor, ensuring that every interaction with the codebase is governed by clear levels of authority.

Role	Responsibility	Authority	Primary Artifacts
Operator (Human)	Strategic ownership and final validation.	Owns commits, pushes, merges,	SoP.md validation, PR merges.

		and secrets.	
Integrator (Lead AI)	Structural integrity and governance maintenance.	Detects system "Drift," generates Dispatch Packets.	TASK.md (Top), ops/ auditing.
Contractor (Guest AI)	Technical execution and implementation drafting.	Operates within defined scope; never commits to main.	TASK.md (Log), storage/handoff/ diffs.

The Operator serves as the ultimate arbiter, holding exclusive rights over the production state and sensitive secrets. This human-in-the-loop requirement is not merely a safety measure but a fundamental constitutional invariant. The Integrator, acting as the lead AI, provides the continuity necessary for long-running projects by monitoring for structural anomalies—referred to as "Drift"—and ensuring that all proposed changes align with the coordinates established in TRUTH.md. The Contractor represents the transient workforce, assigned specific, scoped tasks that must be validated by the Integrator before reaching the Operator for final commitment.[1]

Behavioral Logic and Linguistic Precision

A unique aspect of Stela's governance is the behavioral logic standard imposed on AI agents. To ensure technical auditing and rhythmic precision, agents are prohibited from using contractions. This rule serves to normalize the linguistic density of the repository's discourse, making it easier for automated tools and subsequent agents to parse logic without the ambiguity introduced by colloquial syntax. Furthermore, the platform mandates quantitative reporting. If an execution path deviates, the agent must report the deviation in precise percentages (e.g., "14.7%") rather than using subjective or qualitative descriptions. This requirement for absolute literalism minimizes the risk of misinterpreted instructions and reinforces the system's commitment to objective truth.[1]

The operational directive of equilibrium maintenance further strengthens this framework. A task is considered complete only when the system state reaches equilibrium and the State of Play (SoP.md) is updated. This ensures that the repository never leaves a session in a fragmented state. Agents are also bound by a "Reuse-First Discipline," which requires them to cross-reference all new proposals against existing templates in the ops/ directory, preventing the proliferation of redundant or conflicting tools.[1]

The Filing Doctrine and Repository Hygiene

The structural organization of Stela is governed by the 4-Line Heuristic, a classification

system that ensures every file has a definitive jurisdictional home. This filing doctrine, documented in TRUTH.md, categorizes files based on their functional role within the platform.[1]

1. **System Run Logic:** Files that drive the system live in the ops/ directory. This includes binaries and the operational registry.
2. **System Explainability:** Files that provide documentation and rationale live in the docs/ tree.
3. **Work Payloads:** The actual product code resides in the projects/ directory.
4. **Transient Data:** Temporary artifacts, logs, and caches are relegated to the storage/ directory, which is traditionally ignored by version control except for essential scaffolding.

This doctrine prevents the commingling of platform logic and project payloads, a common source of entropy in aging codebases. By maintaining a clear "Map" of the repository structure in docs/library/MAP.md, the platform provides agents with the necessary terrain awareness to operate without inventing new, non-canonical paths.[1]

Verification Gates and Policing Workflows

Stela implements automated policing via GitHub Actions to ensure adherence to these constitutional rules. The repo-gates.yml workflow enforces tree verification and truth linting on every push and pull request. Specifically, the tools/verify_tree.sh script ensures that the repository skeleton remains intact and that storage drift is kept to a minimum. The tools/lint_truth.sh script scans for forbidden spellings of the platform name (e.g., "Steela" or "Stella") and polices governance terminology across authored surfaces.[1]

Crucially, the sop_policing.yml workflow enforces the "Canon Update Rule." Any change to TRUTH.md, TASK.md, or the docs/ library must be accompanied by an entry in SoP.md that explains what changed and why. This prevents the loss of institutional knowledge and ensures that the repository's history remains as auditable as its current state. Similarly, task_policing.yml monitors changes to critical gate surfaces like ops/bin/open or docs/library/MANUAL.md, requiring a corresponding update to TASK.md to reflect new operational behaviors.[1]

The Operational Runtime: Subsystem Analysis

The operational capabilities of Stela are driven by a set of binaries and libraries located in the ops/ directory. These tools manage the lifecycle of a task, from initial context rehydration to the final proof bundle generation.

Context Rehydration and Session Initialization

The ops/bin/open tool serves as the primary entry point for a new session. It generates an "Open Prompt" that includes freshness gates—checking the active branch, HEAD hash, and

current intent. This process rehydrates the agent by providing the necessary context from canonical pointers like AGENTS.md and llms.txt. The output includes a summary of the git state, ensuring that the worker is aware of the current working tree status (clean or dirty) before beginning work.[1]

Complementing this is the ops/bin/dump tool, which serializes the platform state for agent ingestion. The dump supports different scopes—"platform," "full," or "project"—and can output in various formats, including one optimized for ChatGPT. The tool generates a manifest that indexes all included files, providing their sizes and SHA256 hashes for auditability. This serialization process is critical for "Zero-Memory" agents, as it provides a complete, deterministic snapshot of the required files without requiring the agent to perform expensive or imprecise crawls of the file system.[1]

The Projects Runtime Contract and Orchestration

For complex, multi-stage tasks, Stela utilizes an integration contract implemented via the ops/lib/project/INTEGRATION.md runtime. This contract defines how the system composes tasks, agents, and skills into a deterministic, auditable execution flow. The integration layer does not define agent behavior but sequences it, enforcing structured handoffs between specialized roles such as the planner, code-reviewer, and security-reviewer.[1]

Workflow Type	Agent Sequence	Purpose
Feature	planner -> code-reviewer -> security-reviewer	Comprehensive implementation and auditing.
Bugfix	explorer -> code-reviewer	Root cause analysis and verification.
Refactor	architect -> code-reviewer	Structural improvement without logic change.
Security	security-reviewer -> code-reviewer -> architect	Vulnerability remediation and design hardening.

This orchestration model ensures that no single agent stance is considered sufficient for high-stakes changes. Each step in the sequence produces a "Handoff Artifact" containing findings, modified files, and open questions for the next agent in the chain. The final output is an "Integration Report" with a clear recommendation: SHIP, NEEDS WORK, or BLOCKED.[1]

The Skills Subsystem: Heuristics and Knowledge Harvesting

One of the most innovative components of Stela is its Skills subsystem, which provides a structured mechanism for capturing and reusing technical workflows. Skills are on-demand guidance packages for production payloads, kept separate from the platform's core context to prevent "Context Hazard"—the bloating of an agent's reasoning space with irrelevant procedures.[1]

Heuristic-Driven Provenance Capture

The ops/lib/skill/heuristics.sh engine provides the forensic power behind the Skills subsystem. It analyzes git diffs and logs to detect "Hot Zones" (areas of high line volume change) and "High Churn" (files touched most frequently). This data is used to generate a provenance block for every new skill candidate, capturing the exact timestamp, DP-ID, branch, and HEAD hash where the lesson was learned. By anchoring skills in concrete git evidence, Stela avoids the "Faulty Memory" problem often associated with human documentation.[1]

The skill_lib.sh utility facilitates the lifecycle of a skill from harvest to promotion. During the "Harvest" phase, the tool auto-detects provenance and uses the heuristics engine to suggest a name or context. It also performs a "Semantic Collision Check" by comparing keywords in the new candidate against existing skills in docs/library/skills/, preventing the creation of redundant documentation. If a potential collision is detected, the operator must provide a --force flag to proceed, ensuring a high signal-to-noise ratio in the library.[1]

Promoted Skills and Procedural Rigor

Once a skill is promoted, it is registered in docs/library/INDEX.md and becomes available for explicit invocation in a Dispatch Packet. Each skill follows a strict template, including scope, invocation guidance, drift preventers, and a procedure.

Skill ID	Title	Key Friction Context	Primary Procedure
S-LEARN-01	Verification Loop	Universal payload verification.	Clean state -> Lint -> Build -> Test -> Diff.
S-LEARN-02	Stela Stack Guidelines	High churn architecture decisions.	Adhere to Next.js 14 / FastAPI 3.11 stack.

S-LEARN-03	Zenith Reference Pattern	API integration / data fetching logic.	Implement Zenith Envelope and Hook.
S-LEARN-04	Coding Standards	Syntax and formatting drift.	Mandatory type hints, Pydantic, Black.
S-LEARN-05	Security Protocols	Auth and data access permissions.	Supabase RLS mandate, Zod validation.
S-LEARN-06	Advanced Git Forensics	Legacy code friction analysis.	Use numstat and churn detection logs.
S-LEARN-07	Harvest/Promote Skills	Task closeout knowledge loss.	Use skill_lib.sh to capture DP lessons.

The "Drift Preventers" section of each skill provides critical "Stop Conditions" for agents. For example, in S-LEARN-03, an agent must stop if an endpoint returns a raw array or bare primitive instead of the mandated Zenith Envelope. This proactive defense against paradigm mixing is a hallmark of the Stela approach to technical debt management.[1]

Emerging Documentation Standards: The llms.txt Specification

As Stela seeks to further optimize its interactions with external AI tools and specialized coding assistants, the emerging llms.txt and llms-full.txt standards present a significant opportunity. Proposed as a "robots.txt for reasoning engines," this specification provides a machine-readable entry point for LLMs to understand documentation structure and content without the overhead of parsing unstructured HTML or navigating complex directory trees.[2]

The Core Philosophy of llms.txt

The llms.txt format is designed to solve several challenges in the modern AI ecosystem. Traditional documentation is optimized for human consumption, often featuring sidebars, navigation menus, and JavaScript-rendered elements that consume valuable tokens and can lead to model hallucinations. By serving structured Markdown at /llms.txt, a site provides AI crawlers with a clear, curated outline of its most important content.[2]

The specification differentiates between three levels of information:

- **llms.txt**: A curated navigation file acting as an "Executive Summary." It contains an H1 title, a blockquote summary, and links to the most relevant resources, organized into logical sections.[2]
- **llms-small.txt**: An optional abbreviated version for smaller context windows, providing only the most critical content.[7]
- **llms-full.txt**: A comprehensive file that compiles the entire documentation set into a single Markdown document, facilitating "Zero-Hop" context loading.[5]

The adoption of this standard has been accelerated by platforms like Mintlify and GitBook, which now automatically generate these files for thousands of technical repositories, including high-profile projects like Anthropic, Cursor, and Supabase.[2]

The Role of llms-full.txt in Content Retrieval

The llms-full.txt file is particularly relevant for the Stela platform's "Zero-Memory" rehydration needs. By concatenating all canonical rationale and procedures into a single stream, it allows an agent to ingest the system's logic in one request. This approach is complementary to both Retrieval-Augmented Generation (RAG) and Context-Augmented Generation (CAG). While RAG searches for information and CAG uses pre-loaded info, llms-full.txt provides the exact material needed for the reasoning engine to ground its answers in authoritative, up-to-date documentation.[6]

However, the scale of these files can become a constraint. Vercel's llms-full.txt has been described as a "400,000-word novel," and Cloudflare's version exceeds 3.7 million tokens. For many current models, this exceeds the context window limitations, requiring IDEs like Cursor to chunk and index the content locally using RAG strategies.[9]

Strategic Evaluation: Would llms-full.txt Benefit Stela?

The question of whether Stela should implement the llms-full.txt standard must be viewed through the lens of its existing constitutional goals: explainability, provenance, and equilibrium. The current Stela configuration already includes a placeholder llms.txt used as a pointer map.[1] Expanding this into a full-context artifact offers several high-order benefits and a few manageable risks.

Benefit 1: Zero-Maintenance Context Rehydration

Stela's current process for rehydrating an agent involves running ops/bin/dump to serialize a set of files defined in a manifest. While precise, this requires the Integrator or Operator to manually manage what constitutes the "Core Set." An automated llms-full.txt generator could serve as a "Self-Updating Manifest." By crawling the docs/ library and canonical files (TRUTH.md, AGENTS.md, SoP.md), the system can produce a noise-free version of its

governance logic that is always in sync with the repository's HEAD hash.[13]

For guest contractors working in advanced IDEs like Cursor or Windsurf, the benefit is immediate. These tools can natively index documentation from a URL. By providing an llms-full.txt endpoint, Stela allows the IDE to provide accurate, context-aware suggestions for the Zeneth hook or the Skills Context Hazard rules without the agent needing to manually open and read the individual .md files in docs/library/skills/.[16]

Benefit 2: Mitigation of Regression Drift

A common friction point in agent-assisted development is the model's reliance on its generic training data, which may be outdated or conflict with project-specific patterns. llms-full.txt provides a "Librarian's List" of the authoritative source material. When an AI assistant is grounded in the full content of Stela's documentation, it is less likely to propose solutions that violate the "4-Line Heuristic" or the "Contraction Prohibition".[5]

The "Friction Context" captured in promoted skills is particularly valuable here. If the llms-full.txt includes the "Trap/Solution" patterns from S-LEARN-XX files, the reasoning engine can understand *why* certain procedures are mandated. This creates a higher level of "Architectural Fidelity," as the agent can reason across the historical rationale of the system's evolution.[1]

Benefit 3: Enhanced Interoperability with Model Context Protocol (MCP)

The industry is moving toward standardized interfaces for AI tools, such as the Model Context Protocol (MCP). MCP servers like the langchain-ai/mcpdoc server specifically leverage llms.txt files to guide LLMs to the most appropriate documentation. By adopting the standard, Stela positions itself to utilize a wide range of off-the-shelf MCP tools for auditing, search, and documentation synthesis, reducing the need for custom internal shell scripts for common tasks.[12]

Risks and Technical Constraints

The primary risk associated with llms-full.txt for Stela is token noise. If the documentation library grows excessively large, the sheer volume of text could degrade the model's instruction-following quality. As instruction count increases, the performance of the model on specific directives often decreases. This is particularly dangerous for a system like Stela, which relies on absolute literalism and rigorous gatekeeping.[20]

Furthermore, there is a risk of "Security Leakage." Stela's docs/ library is designed to explain the system, but care must be taken to ensure that an automated concatenator does not inadvertently include sensitive implementation details that should remain restricted to the projects/ directory or the Operator persona. Any implementation of llms-full.txt in Stela must

strictly respect the 4-Line Heuristic and ignore the projects/ and storage/ trees by default.[1]

Implementation Strategy for Stela-Full-Context

To realize the benefits of the llms.txt standard while mitigating the associated risks, Stela should adopt a tiered, automated context strategy. This strategy integrates with the existing ops/ subsystem and the repo-gates CI/CD pipeline.

Tiered Documentation Artifacts

Stela should generate three distinct files to support different reasoning requirements and context window limitations.

Artifact	Content Scope	Use Case
llms.txt	Navigation index and pointers.	Discovery and quick navigation via mcpdoc.
llms-small.txt	TRUTH.md, AGENTS.md, and top 10 promoted skills.	Lightweight rehydration for routine Contractor tasks.
llms-full.txt	Full docs/ library + SoP.md (active) + all skills.	Architectural planning and comprehensive auditing.

The llms-small.txt is an important addition to the standard specification, tailored for Stela's high-precision requirements. It ensures that the most critical constitutional rules and the most frequently used procedures are always within the model's high-attention zone.[7]

Automated Generation via ops/bin/llms-gen

A new operational binary, ops/bin/llms-gen, should be developed to automate this process. This tool should use the llmsTxtTemplateGetter logic observed in other advanced implementations, pulling titles and descriptions from the frontmatter of documentation files to ensure semantic accuracy. The generation logic must include the following invariants:

- **Noise Filter**: Automatically remove structural Markdown elements that do not contribute to semantic understanding (e.g., table of contents generated for human readers).
- **Redaction Filter**: Run all text through the existing redaction stream used in skill_lib.sh to prevent the accidental leakage of secrets or credentials.[1]
- **Formatting Rule**: Every page content block must be clearly anchored with its repository-relative path and description to provide the LLM with a stable hierarchy.[23]

Integration with State-of-Play and Repo-Gates

To maintain the system's "Provenance-Forward" stance, the generation of these files must be triggered by the repo-gates.yml workflow. Whenever a pull request is merged into main, the llms-full.txt should be regenerated and its hash recorded in the merge commit's metadata. This ensures that the documentation visible to external AI tools is always a perfect reflection of the repository's production state.[13]

The SoP.md ledger should also be updated to reflect changes in the "AI-Readable Surface." By treating the llms.txt variants as canonical artifacts, Stela enforces the same level of accountability for its machine interfaces as it does for its human-facing documents.[1]

Analytical Synthesis of Stela Governance and LLM Ingestion

The integration of llms-full.txt into Stela represents a transformation from a "Pull" context model to a "Push" context model. Currently, agents must "Pull" information by requesting specific files or running a dump. With the new standard, the platform "Pushes" its entire logical structure to the agent's reasoning space through a single, optimized artifact.

Second-Order Impacts on Agentic Autonomy

By providing a full-context Markdown stream, Stela reduces the "Context Gap"—the difference between what the agent knows and what the system requires. This leads to several second-order impacts:

- **Reduced Hallucination in Skill Selection**: An agent can see the entire skills library and understand the semantic boundaries between S-LEARN-01 and S-LEARN-03. This prevents the model from "inventing" a procedure that slightly deviates from the Stela standard.[1]
- **Improved Alignment with Staffing Protocols**: When the model has the full AGENTS.md and docs/GOVERNANCE.md in its primary context, it is more likely to respect the jurisdictional boundaries of the Integrator and Operator. It will "Stop and Ask" more reliably when a task exceeds its scoped Contractor authority.[1]
- **Enhanced Proof Bundle Integrity**: The Mandatory Closing Block requirement in TASK.md is more likely to be followed correctly if the agent can reference multiple "Good Examples" of receipts provided in the llms-full.txt.[1]

Third-Order Impacts on System Equilibrium

In the long term, the adoption of standardized, machine-readable governance artifacts positions Stela as a "Leader in Explainable CMS." As AI agents become the primary users of technical documentation, a repository that provides a high-quality, token-efficient interface

will naturally experience less friction and lower maintenance costs.

The "Ripple Effect" of this integration is the formalization of "Machine-First Documentation." While traditional developers may still read the individual files in docs/, the platform acknowledges that the AI assistant is the first "Reader" of the codebase. By optimizing for this first reader, Stela ensures that the resulting code payloads are higher quality, better typed, and strictly adherent to the project's invariants.[2]

Technical Recommendations and Roadmap

Based on the exhaustive analysis of the Stela repository and the research materials regarding the llms.txt standard, the following recommendations are provided for professional implementation.

Strategic Implementation Phases

1. **Phase 1: Binary Development (Immediate)**: Develop ops/bin/llms-gen. This tool should concatenate TRUTH.md, AGENTS.md, and all files in docs/library/ (including promoted skills). It must respect the 4-Line Heuristic and ignore projects/ to prevent token bloat and security leakage.
2. **Phase 2: Gate Integration (Week 1)**: Add a step to repo_gates.yml to run the generation script. Configure the workflow to fail if the generated files do not adhere to the Markdown structure defined in the llms.txt specification.
3. **Phase 3: Tiered Artifact Exposure (Week 2)**: Update the repository's entry point (llms.txt) to point to the new llms-small.txt and llms-full.txt variants. Register these as canonical context artifacts in ops/lib/manifests/CONTEXT.md.
4. **Phase 4: Tooling Alignment (Week 3)**: Update ops/bin/open and ops/bin/dump to leverage the new artifacts. When an agent requests platform-scope context, the dump tool can opt to provide the llms-full.txt instead of raw directory snapshots, improving rehydration speed and accuracy.

Standardized Format for Stela-Full-Context

The generated llms-full.txt should follow this structure to maximize LLM reasoning:

- **Title and Summary**: H1 and blockquote description from README.md.
- **Constitutional Core**: Full contents of TRUTH.md and AGENTS.md.
- **Operational Procedures**: Concatenated docs/library/MANUAL.md and docs/library/INTEGRATION.md.
- **The Skills Library**: Every S-LEARN-XX file, clearly delineated by headers.
- **The Historical Context**: The most recent 10 entries of SoP.md to provide continuity and "State of Play" awareness.

By including the active history entries, Stela provides the agent with a "Sense of Time," allowing it to understand what was recently shipped and what patterns are currently in favor.

This is a critical factor in preventing "Regression Drift" and ensuring system equilibrium.[1]

Nuanced Conclusions on Machine-Readable Governance

The Stela platform represents the vanguard of provenance-forward engineering. Its commitment to absolute literalism, quantitative reporting, and hierarchical jurisdictional control creates a uniquely stable environment for agentic systems. However, the current "Snap-Shot" approach to context loading, while technically accurate, lacks the semantic fluidity provided by emerging industry standards.

The integration of llms-full.txt is not merely a documentation task; it is the formalization of Stela's governance for the AI era. By providing a single, curated, and forensic ingestion point, Stela ensures that its agents—both transient and permanent—operate with a unified understanding of the system's laws. This leads to a repository that is not only explainable to humans but optimized for the reasoning engines that will build its future.

The strategic benefits of this integration far outweigh the technical costs of implementation. Through tiered artifact generation and automated CI/CD policing, Stela can manage the risk of context bloat while gaining the massive efficiency of "Zero-Maintenance Rehydration." As the repository grows, this machine-readable entry point will become the primary mechanism through which system equilibrium is maintained and architectural drift is defeated..[1]

Works cited

1. dump-full-main-9ab382c78.txt
2. What is llms.txt? Breaking down the skepticism - Mintlify, accessed February 3, 2026, https://www.mintlify.com/blog/what-is-llms-txt
3. LLMs.txt Explained | TDS Archive - Medium, accessed February 3, 2026, https://medium.com/data-science/llms-txt-414d5121bcb3
4. What is LLMs.txt? Complete Guide to Creating LLMs.txt Files for Your Website, accessed February 3, 2026, https://blog.starmorph.com/blog/what-is-llms-txt-file-guide
5. Working with llms.txt | Platform Overview - Mastercard Developers, accessed February 3, 2026, https://developer.mastercard.com/platform/documentation/agent-toolkit/working-with-llmstxt/
6. LLMs.txt: The Emerging Standard Reshaping AI-First Content Strategy | ScaleMath, accessed February 3, 2026, https://scalemath.com/blog/llms-txt/
7. Give it in plain text: Making your content AI-Ready | Sanity, accessed February 3, 2026, https://www.sanity.io/blog/improving-the-agent-experience-for-sanity-learn
8. LLM-Friendly Documentation - Graphite, accessed February 3, 2026, https://graphite.com/docs/ai-ingestion

9. The Complete Guide to llms.txt: Should You Care About This AI Standard? - Publii, accessed February 3, 2026, https://getpublii.com/blog/llms-txt-complete-guide.html
10. How to Create an llms.txt File for Any Website - Firecrawl, accessed February 3, 2026, https://www.firecrawl.dev/blog/How-to-Create-an-llms-txt-File-for-Any-Website
11. DOCS: llms.txt and llms-full.txt for mise.jdx.dev docs, improved LLM usage #5945 - GitHub, accessed February 3, 2026, https://github.com/jdx/mise/discussions/5945
12. llms.txt - GitHub Pages, accessed February 3, 2026, https://langchain-ai.github.io/langgraph/llms-txt-overview/
13. Best llms.txt implementation platforms for AI-discoverable APIs in January 2026 - Fern, accessed February 3, 2026, https://buildwithfern.com/post/best-llms-txt-implementation-platforms-ai-discoverable-apis
14. llms-txt-action - Marketplace - GitHub, accessed February 3, 2026, https://github.com/marketplace/actions/llms-txt-action
15. We made our docs AI-ready. Here's why it matters for your next build. - Prepr, accessed February 3, 2026, https://prepr.io/blog/we-made-our-docs-ai-ready-heres-why-it-matters-for-your-next-build
16. What's the best way to add context to Cursor without adding files to the codebase? - Reddit, accessed February 3, 2026, https://www.reddit.com/r/cursor/comments/1q7g1nt/whats_the_best_way_to_add_context_to_cursor/
17. Teaching Your LLM About DuckDB the Right Way: How to Fix Outdated Documentation, accessed February 3, 2026, https://motherduck.com/blog/fix-outdated-llm-documentation-duckdb/
18. Using AI coding assistants - Modular Docs, accessed February 3, 2026, https://docs.modular.com/max/coding-assistants/
19. llms-full.txt - assistant-ui, accessed February 3, 2026, https://www.assistant-ui.com/llms-full.txt
20. llms-full.txt - Ona, accessed February 3, 2026, https://ona.com/docs/llms-full.txt
21. Crawl LLMs.txt for better efficiency - Archon - oTTomator Community, accessed February 3, 2026, https://thinktank.ottomator.ai/t/crawl-llms-txt-for-better-efficiency/6657
22. llms-full.txt - Bun, accessed February 3, 2026, https://bun.sh/llms-full.txt
23. plugin-llms - VuePress Ecosystem, accessed February 3, 2026, https://ecosystem.vuejs.press/plugins/ai/llms.html
24. LLM-Friendly Documentation: Creating Content That AI Can Understand and Process Effectively - ARON HACK, accessed February 3, 2026, https://aronhack.com/llm-friendly-documentation-creating-content-that-ai-can-understand-and-process-effectively/